ZSH

CHEAT SHEETS

A Complete Reference for the Z Shell

Startup Files · Options · Quoting · Redirection · History · ZLE · Parameter Expansion · Arrays · Globbing · Glob Qualifiers · Functions · Prompt · Completion · Plugins

27 REFERENCE SHEETS

CONTENTS

SETUP & CONFIGURATION

01 Introduction & Startup Files
02 Shell Options

THE COMMAND LINE

03 Output, Quoting & Escaping
04 Redirection & Pipes
05 History & History Expansion
06 Line Editor (ZLE) & Key Bindings

VARIABLES & PARAMETERS

07 Variables & typeset
08 Special Parameters
09 Parameter Expansion
10 Parameter Expansion Flags

ARRAYS

11 Arrays
12 Associative Arrays

GLOBBING

13 Globbing Basics
14 Glob Qualifiers & Modifiers

TESTS, ARITHMETIC & CONTROL FLOW

15 Test Expressions
16 if, case & select
17 Arithmetic
18 Loops

FUNCTIONS & SCRIPTING

19 Functions
20 Scripts, Errors & Debugging
21 Jobs & Process Substitution

INTERACTIVE CUSTOMIZATION

22 Aliases & Directory Navigation
23 Prompt & Hooks
24 Completion System
25 Plugins, Frameworks & Performance

QUICK REFERENCE

26 Quick Reference — Core Syntax
27 Quick Reference — Advanced

ZSH

Table of Contents

27 sheets · syntax · examples · notes

SETUP & CONFIGURATION 01 – 02

- 01 Introduction & Startup…
- 02 Shell Options

THE COMMAND LINE 03 – 06

- 03 Output, Quoting & Esc…
- 04 Redirection & Pipes
- 05 History & History Expa…
- 06 Line Editor (ZLE) & Key…

VARIABLES & PARAMETERS 07 – 10

- 07 Variables & typeset
- 08 Special Parameters
- 09 Parameter Expansion
- 10 Parameter Expansion F…

ARRAYS 11 – 12

- 11 Arrays
- 12 Associative Arrays

GLOBBING 13 – 14

- 13 Globbing Basics
- 14 Glob Qualifiers & Modi…

TESTS, ARITHMETIC & CONTROL FLOW 15 – 18

- 15 Test Expressions
- 16 if, case & select
- 17 Arithmetic
- 18 Loops

FUNCTIONS & SCRIPTING 19 – 21

- 19 Functions
- 20 Scripts, Errors & Debu…
- 21 Jobs & Process Substi…

INTERACTIVE CUSTOMIZATION 22 – 25

- 22 Aliases & Directory Na…
- 23 Prompt & Hooks
- 24 Completion System
- 25 Plugins, Frameworks & …

QUICK REFERENCE 26 – 27

- 26 Quick Reference — Co…
- 27 Quick Reference — Ad…

Introduction & Startup Files

WHAT ZSH IS

Zsh (the Z shell) is a Unix shell written by Paul Falstad in 1990. It runs almost all everyday `sh`/`bash` commands unchanged and adds recursive globbing, parameter expansion flags, a programmable completion system, and themeable prompts. It has been the **default login shell on macOS since Catalina** (10.15, 2019). Its behavior is configured by dotfiles in `$HOME` (or `$ZDOTDIR`), read in a fixed order that depends on the **type** of shell being started.

GOOD TO KNOW

- `$ZSH_VERSION` holds the running version, e.g. `5.9`
- `$SHELL` is your *login* shell, not the current one. Use `echo $0` or `ps -p $$`
- System-wide copies run first: `/etc/zshenv`, `/etc/zprofile`, `/etc/zshrc`
- Zsh does **not** read `.bashrc` or `.bash_profile`
- Check the shell type with `[[ -o login ]]` or `[[ -o interactive ]]`

INSTALL, SWITCH & RUN

```
brew install zsh                # newer zsh on macOS (system one is /bin/zsh)
sudo apt install zsh            # Debian / Ubuntu
zsh --version                   # zsh 5.9 (x86_64-apple-darwin23.0)
which zsh                       # /bin/zsh
chsh -s "$(which zsh)"          # make zsh your login shell, then log out/in
echo $ZSH_VERSION               # 5.9

# hello.zsh
#!/usr/bin/env zsh
print "Hello, World"            # Hello, World

zsh hello.zsh                   # run with zsh explicitly
chmod +x hello.zsh && ./hello.zsh
zsh -f                          # clean shell: skip all startup files
exec zsh                        # replace current shell, re-read rc files
```

CODE EXPLAINED

→ `chsh -s` — changes the login shell; a non-system zsh must be listed in `/etc/shells` first

→ `#!/usr/bin/env zsh` — finds zsh on `$PATH`, so it works with Homebrew zsh too

→ `print` — zsh's builtin for output (see Sheet 03)

→ `zsh -f` — same as `NO_RCS`: the fastest way to tell if a problem comes from your config

→ `exec zsh` — cleaner than `source ~/.zshrc`, which keeps stale aliases

STARTUP FILES

FILE	READ WHEN	PUT HERE
`~/.zshenv`	every shell, incl. scripts	`ZDOTDIR`, env vars; no output
`~/.zprofile`	login shells, before .zshrc	PATH, `brew shellenv`
`~/.zshrc`	interactive shells	aliases, options, prompt, completion
`~/.zlogin`	login shells, after .zshrc	rarely needed
`~/.zlogout`	login shell exits	cleanup, `clear`

WHICH FILES RUN

SHELL TYPE	ORDER
`login + interactive`	zshenv → zprofile → zshrc → zlogin
`interactive only`	zshenv → zshrc
`script (non-interactive)`	zshenv
`zsh -l -c 'cmd'`	zshenv → zprofile → zlogin

⚠ COMMON MISTAKES

- Printing output in `.zshenv`: it runs for every script and `scp` session, and stray output breaks them
- Expecting scripts to see `.zshrc` aliases: non-interactive shells only read `.zshenv`
- Editing `.bash_profile` after switching: zsh ignores it
- `chsh` refusing Homebrew zsh: add its path to `/etc/shells` first

✓ TIPS & BEST PRACTICES

- On macOS, set PATH in `.zprofile`: `/etc/zprofile` runs `path_helper`, which reorders PATH set in `.zshenv`
- Move your dotfiles with `export ZDOTDIR=~/.config/zsh` in `~/.zshenv`
- Debug slow or broken startup with `zsh -f`, then add files back one at a time

QUICK SUMMARY

- `$ZSH_VERSION` — running version
- `chsh -s $(which zsh)` — set login shell
- `.zshrc` — interactive config
- `.zprofile` — login-only setup like PATH
- `zsh -f` — start with no config

Shell Options

OPTIONS

Shell options are named on/off switches that change how zsh behaves. `setopt NAME` turns one on and `unsetopt NAME` turns it off. Names are **case-insensitive and ignore underscores**, so `AUTO_CD`, `autocd` and `AutoCd` are the same option. A `NO_` prefix inverts an option: `setopt NO_BEEP` is the same as `unsetopt BEEP`.

NOTES

- `setopt` with no args lists options that differ from the default
- `set -o` lists every option with on/off
- `[[ -o name ]]` tests whether an option is set
- `$options[name]` reads `on` / `off` from an associative array

SETTING & SCOPING OPTIONS

```
setopt AUTO_CD                # type a directory name to cd into it
setopt EXTENDED_GLOB NO_BEEP  # several at once
unsetopt CORRECT              # turn off spelling correction
setopt NO_CLOBBER             # > refuses to overwrite existing files
[[ -o autocd ]] && echo on    # on   (case and _ ignored)
echo $options[autocd]         # on

tidy() {
  emulate -L zsh              # zsh defaults; LOCAL_OPTIONS turned on
  setopt EXTENDED_GLOB        # undone automatically on return
  print -l *.txt~draft*       # all .txt except draft*
}

emulate sh -c 'source old.sh' # run sh code with sh rules
```

CODE EXPLAINED

→ `emulate -L zsh` — resets options to zsh defaults and sets `LOCAL_OPTIONS` so changes end with the function

→ `LOCAL_OPTIONS` — restores the caller's options when a function returns

→ `emulate sh -c` — runs one command (or file) under sh rules, then switches back

→ `~` in `*.txt~draft*` — "except" pattern; needs `EXTENDED_GLOB` (Sheet 13)

POPULAR INTERACTIVE OPTIONS

OPTION	EFFECT
`AUTO_CD`	`dir` alone runs `cd dir`
`AUTO_PUSHD`	`cd` pushes onto the dir stack
`CORRECT`	offer to fix mistyped commands
`INTERACTIVE_COMMENTS`	allow `#` comments at the prompt
`NO_BEEP`	silence the terminal bell

USEFUL IN SCRIPTS

OPTION	EFFECT
`EXTENDED_GLOB`	enables `^` `~` `#` patterns
`NULL_GLOB`	no match → empty, not error
`NO_UNSET`	error on unset variables
`ERR_EXIT`	exit on first failing command
`PIPE_FAIL`	pipeline fails if any part fails

EMULATE

COMMAND	EFFECT
`emulate zsh`	zsh default options
`emulate sh / ksh / csh`	act like that shell
`emulate -L zsh`	local to the current function
`emulate -R sh`	reset *all* options to sh mode
`emulate`	print current mode

⚠ COMMON MISTAKES

- Setting `EXTENDED_GLOB` globally, then being surprised `^` and `#` act as glob operators in filenames
- Changing options inside a function without `LOCAL_OPTIONS`: the change leaks into the whole shell
- Assuming a bash option name exists: zsh has no `shopt`, so use `setopt` instead

✓ TIPS & BEST PRACTICES

- Start every reusable function with `emulate -L zsh` so the user's options can't break it
- Find an option with `setopt | grep -i hist` or `man zshoptions`
- Turn on `INTERACTIVE_COMMENTS` so pasted snippets with `# comments` work at the prompt

QUICK SUMMARY

- `setopt X` / `unsetopt X` — on / off
- `NO_X` — inverted form
- `emulate -L zsh` — safe defaults in functions
- `[[ -o X ]]` — test an option
- `set -o` — show all options

Output, Quoting & Escaping

PRINTING & QUOTING

Zsh has three output commands: `echo` (portable, but zsh's version **interprets escapes like `\t` by default**), `print` (zsh's own, with options for lines, columns, prompts and stderr), and `printf` (C-style formatting). Quoting controls what the shell expands: **single quotes** keep everything literal, **double quotes** expand `$vars` and `$(cmds)` but stop word splitting and globbing, and `$'...'` turns escape sequences into real characters.

NOTES

- Unlike bash, zsh does **not** word-split `$var` unquoted, so `echo $file` is safe with spaces
- `print -r` and `echo -E` print backslashes literally
- `--` ends options: `print -r -- "$x"` is safe when `$x` starts with `-`

OUTPUT COMMANDS

```
echo "Hello"                  # Hello
echo "a\tb"                   # a    b    (zsh echo expands escapes)
echo -E "a\tb"                # a\tb   (-E: raw)
print -l one two three        # one / two / three on separate lines
print -n "prompt> "           # no newline
print -P "%F{green}ok%f"      # ok in green (prompt escapes)
print -r -- "$raw"            # raw, no escape processing
print -u2 "error!"            # to stderr, same as >&2
print -a -C 3 a b c d e f     # a b c / d e f   (3 columns)
printf "%-6s|%5.2f\n" id 3.14159   # id     | 3.14
printf "%s\n" a b c           # format repeats: a / b / c

name="Ada"
echo 'Hi $name'               # Hi $name    (literal)
echo "Hi $name"               # Hi Ada      (expanded)
echo $'tab:\there'            # tab:    here (ANSI-C quoting)
echo "cost: \$5"              # cost: $5
v='a b*'; echo ${(q)v}        # a\ b\*      (quoted for reuse)
```

CODE EXPLAINED

- → `print -l` — one argument per line; great for arrays: `print -l $path`
- → `print -P` — expands prompt escapes like `%F{red}`, `%B`, `%~` (Sheet 23)
- → `print -u2` — writes to file descriptor 2 (stderr)
- → `printf` — reuses the format until all arguments are consumed
- → `${(q)v}` — backslash-quotes a value so it can be pasted back into a command

QUOTING FORMS

FORM	EXPANDS
`'text'`	nothing; fully literal
`"text"`	`$var`, `$(cmd)`, `$((math))`
`$'text'`	escapes like `\n` `\t` `\e`
`\x`	one literal character
`${(q)v}`	backslash-quoted value
`${(qq)v}`	single-quoted value

ESCAPE SEQUENCES

ESCAPE	MEANING
`\n` `\t`	newline, tab
`\e` `\a`	escape (ANSI colors), bell
`\\`	backslash
`\u2713`	Unicode ✓
`\x41`	hex byte → A
`\c`	stop output (echo/print)

PRINT OPTIONS

FLAG	EFFECT
`-l`	one argument per line
`-n`	no trailing newline
`-r`	raw, ignore escapes
`-P`	prompt escapes
`-u N`	write to fd N
`-f fmt`	printf-style format

⚠ COMMON MISTAKES

- Using `echo "$x"` where `$x` may start with `-n` or `-e`: it's read as an option. Use `print -r -- "$x"`
- Expecting `'$HOME'` to expand: single quotes are always literal
- Writing `echo "\n"` and expecting the bash result `\n`: zsh's echo prints a real newline
- Putting `'` inside single quotes: end and reopen, `'it'\''s'`, or use `$'it\'s'`

✓ TIPS & BEST PRACTICES

- Prefer `print` or `printf` in zsh scripts; `echo` behavior varies across shells
- Send errors to stderr with `print -u2` so pipelines only carry data
- Use `${(q)var}` when building a command string to `eval` or show to a user
- Color output portably with `print -P '%F{red}...%f'` instead of raw `\e[31m` codes

QUICK SUMMARY

- `print -l` — one item per line
- `printf` — formatted output
- `'…'` — literal, `"…"` — expands vars
- `$'…'` — real escape characters
- `${(q)v}` — safely quoted value

Redirection & Pipes

REDIRECTION

Every command has three standard streams: **stdin (0)**, **stdout (1)** and **stderr (2)**. Redirections point them at files, and pipes connect one command's stdout to the next command's stdin. Zsh adds **MULTIOS**: redirect one stream to several files at once (an implicit `tee`), or read several files in a row (an implicit `cat`). Redirections are processed left to right, so their order matters.

NOTES

- `MULTIOS` is on by default
- `NO_CLOBBER` protects files from `>` ; `>|` forces it
- `$pipestatus` holds the exit code of every command in the last pipeline
- `< file` with no command shows the file with `$READNULLCMD` (default `more`)

REDIRECTION IN PRACTICE

```
ls > files.txt                # stdout to file (truncates)
ls >> files.txt               # append
sort < names.txt              # stdin from file
cmd 2> err.log                # stderr only
cmd > out.log 2>&1            # stdout + stderr (order matters)
cmd &> all.log                # same thing, shorter
cmd |& grep warn              # pipe stdout + stderr
setopt NO_CLOBBER
echo hi > old.txt             # zsh: file exists: old.txt
echo hi >| old.txt            # force overwrite
date > a.txt > b.txt          # MULTIOS: writes both files
cat < a.txt < b.txt           # MULTIOS: reads a then b
false | true; echo $pipestatus  # 1 0
cat <<EOF                     # here-doc: $vars expand
Home: $HOME
EOF
wc -w <<< "one two three"     # 3   (here-string)
```

CODE EXPLAINED

- `2>&1` — "send 2 where 1 points *now*"; `2>&1 > file` leaves stderr on the terminal
- `|&` — shorthand for `2>&1 |`
- `> a > b` — MULTIOS copies output to both files, like `tee`
- `<<EOF` — lines up to `EOF` become stdin; quote it (`<<'EOF'`) to stop expansion
- `<<<` — a single string as stdin, with a trailing newline

REDIRECTION OPERATORS

SYNTAX	MEANING
`> file`	stdout, truncate
`>> file`	stdout, append
`< file`	stdin from file
`2> / 2>>`	stderr, truncate / append
`&> / &>>`	stdout + stderr
`>| file`	overwrite despite NO_CLOBBER
`<> file`	open read + write
`3>&-`	close fd 3

HERE-DOCS & STRINGS

SYNTAX	MEANING
`<<EOF`	here-doc, expands
`<<'EOF'`	here-doc, literal
`<<-EOF`	strips leading **tabs**
`<<< "$s"`	here-string
`exec 3> log`	open fd 3 for the session

PIPES

SYNTAX	MEANING
`a | b`	stdout of a → stdin of b
`a |& b`	stdout + stderr → b
`$pipestatus`	exit codes, e.g. `(1 0)`
`setopt PIPE_FAIL`	$? = last failure
`cmd | tee f`	screen + file

⚠ COMMON MISTAKES

- Writing `cmd 2>&1 > file` : stderr was copied before stdout moved, so errors still hit the screen
- Running `sort f > f` : the shell truncates `f` before `sort` reads it. Write to a temp file
- Using `<<-EOF` with spaces: only leading **tabs** are stripped
- Checking `$?` after a pipe and missing an early failure: use `$pipestatus` or `PIPE_FAIL`

✓ TIPS & BEST PRACTICES

- Use `&>` for "everything to a file"; it reads better than `> f 2>&1`
- Turn on `NO_CLOBBER` interactively to protect files from accidental `>`
- Use MULTIOS (`> log > /dev/tty`) instead of `tee` for quick double output
- Quote the here-doc delimiter (`'EOF'`) when writing scripts or code that contain `$`

QUICK SUMMARY

- `> >> <` — out, append, in
- `2>&1` / `&>` — merge stderr
- `|&` — pipe both streams
- `<<EOF` / `<<<` — here-doc / here-string
- `$pipestatus` — all pipeline exit codes

History & History Expansion

COMMAND HISTORY

Zsh keeps history in memory (`HISTSIZE` lines) and saves it to `HISTFILE` (`SAVEHIST` lines). Plain zsh has `SAVEHIST=0`, which means **nothing is saved until you set it** (macOS's `/etc/zshrc` sets it for you). **History expansion** with `!` reuses earlier commands or arguments, and `^old^new` reruns the last command with a fix. Search interactively with **Ctrl-R**.

NOTES

- `history` is `fc -l`, so it takes the same flags
- `fc` with no args opens the last command in `$EDITOR`
- `HIST_VERIFY` shows the expansion so you can edit it before running
- `EXTENDED_HISTORY` saves timestamps and durations
- Turn off `!` expansion with `setopt NO_BANG_HIST`

CONFIGURE & USE HISTORY

```
HISTFILE=~/.zsh_history        # where history is saved
HISTSIZE=50000                 # lines kept in memory
SAVEHIST=50000                 # lines written to HISTFILE
setopt SHARE_HISTORY           # share live across open terminals
setopt HIST_IGNORE_ALL_DUPS    # remove older duplicates
setopt HIST_IGNORE_SPACE       # " cmd" (leading space) not saved
setopt EXTENDED_HISTORY        # save start time + duration
setopt HIST_VERIFY             # show !! expansion before running

sudo !!                        # rerun last command with sudo
mkdir projects && cd !$        # !$ = last word of previous command
^teh^the                       # rerun last command, first teh → the
history -10                    # list the last 10 entries
history -i -5                  # last 5 with ISO timestamps
fc                             # edit last command in $EDITOR, then run
```

CODE EXPLAINED

- → `SHARE_HISTORY` — every shell reads and appends history as you go
- → `HIST_IGNORE_SPACE` — prefix a command with a space to keep secrets out of history
- → `!$` — last argument of the previous command; `!*` is all of them
- → `^a^b` — quick substitution; `!!:gs/a/b/` replaces every match
- → `history -i` — needs `EXTENDED_HISTORY` for real timestamps

EVENT DESIGNATORS

SYNTAX	REFERS TO
`!!`	last command
`!n`	history entry n
`!-n`	n commands back
`!str`	latest starting with str
`!?str?`	latest containing str

WORD DESIGNATORS

SYNTAX	WORD
`!$`	last word
`!^`	first argument
`!*`	all arguments
`!:2`	word 2
`!:2-4`	words 2 to 4

MODIFIERS

SYNTAX	EFFECT
`:h :t`	dirname, basename
`:r :e`	drop / keep extension
`:s/a/b/`	replace first a with b
`:gs/a/b/`	replace all
`:p`	print, don't run

⚠ COMMON MISTAKES

- Setting `HISTSIZE` but not `SAVEHIST`: history vanishes when the shell closes
- Making `SAVEHIST` larger than `HISTSIZE`: only `HISTSIZE` lines exist to save
- Using `!` inside double quotes, like `echo "hi!"`: zsh tries history expansion. Use single quotes
- Combining `SHARE_HISTORY` with `INC_APPEND_HISTORY`: they overlap, so pick one

✓ TIPS & BEST PRACTICES

- Press **Ctrl-R** and type a fragment to search history; press it again for older matches
- Bind Up/Down to `up-line-or-search` to search by the typed prefix (Sheet 06)
- Add `:p` to preview: `!git:p` prints the command without running it
- Use `fc -l 1 | grep docker` to search the whole history

QUICK SUMMARY

- `HISTFILE` `HISTSIZE` `SAVEHIST` — set all three
- `!!` `!$` `!*` — reuse commands and args
- `^old^new` — fix and rerun
- `history -10` — list recent entries
- `fc` — edit and rerun

Sheet 06

Line Editor (ZLE) & Key Bindings

bindkey -e / -v · bindkey '^R' · zle -N custom widgets · edi

THE ZSH LINE EDITOR

ZLE is the editor behind the prompt. Every action it can take (moving the cursor, searching history, completing) is a named **widget**, and `bindkey` maps key sequences to widgets in a **keymap**. Zsh ships **emacs** (default) and **vi** keymaps. You can write your own widgets as shell functions that edit the special variables $BUFFER (the whole line) and $CURSOR (cursor position), then register them with `zle -N`.

NOTES

- Find a key's code: press **Ctrl-V** then the key, or run `cat -v`
- `^X` means Ctrl-X; `\e` or `^[` means Esc/Alt
- `bindkey '^R'` shows what one key does
- `zle -al` lists every widget

BINDINGS & CUSTOM WIDGETS

```
bindkey -e                    # emacs keymap (default)
bindkey -v                    # vi keymap
bindkey '^R' history-incremental-search-backward
bindkey '^[[A' up-line-or-search    # Up arrow: search by prefix
bindkey -s '^G' 'git status\n'      # key types a string

# Ctrl-X Ctrl-E: edit the current line in $EDITOR
autoload -Uz edit-command-line
zle -N edit-command-line
bindkey '^X^E' edit-command-line

# Esc Esc: put "sudo " in front of the line
sudo-line() { BUFFER="sudo $BUFFER"; CURSOR=$#BUFFER }
zle -N sudo-line
bindkey '\e\e' sudo-line

KEYTIMEOUT=1                  # 10 ms Esc delay (snappier vi mode)
```

CODE EXPLAINED

- → `bindkey KEY WIDGET` — binds in the current main keymap; add `-M vicmd` for another keymap
- → `autoload -Uz` — loads a function shipped with zsh on first use
- → `zle -N name` — turns a shell function into a widget
- → `CURSOR=$#BUFFER` — moves the cursor to the end of the line
- → `KEYTIMEOUT` — in hundredths of a second (default 40 = 0.4 s)

DEFAULT EMACS KEYS

KEY	WIDGET
`^A ^E`	start / end of line
`^W`	delete word backward
`^U`	delete whole line
`^K`	delete to end of line
`^R`	search history backward
`Alt-.`	insert last word
`^Q / Alt-q`	push-line: stash, run another

WIDGET VARIABLES

NAME	HOLDS
`$BUFFER`	entire command line
`$LBUFFER`	text left of cursor
`$RBUFFER`	text right of cursor
`$CURSOR`	cursor offset
`zle -M 'msg'`	message under prompt
`zle reset-prompt`	redraw the prompt

KEYMAPS

NAME	USED FOR
`emacs`	default editing
`viins`	vi insert mode
`vicmd`	vi command mode
`bindkey -l`	list keymaps
`bindkey -M vicmd`	bind in a keymap

⚠ COMMON MISTAKES

- Binding arrow keys with codes from another terminal: check yours with Ctrl-V, or use `$terminfo[kcuu1]`
- Defining a widget function but forgetting `zle -N` : `bindkey` reports `No such widget`
- Turning on vi mode and losing Ctrl-R: `bindkey -v` replaces the emacs bindings, so rebind it in `viins`
- Leaving `KEYTIMEOUT` at default in vi mode: Esc waits 0.4 s before switching modes

✓ TIPS & BEST PRACTICES

- Ctrl-X Ctrl-E (`edit-command-line`) is the easiest way to edit long one-liners
- Alt-. repeats the last argument; press again for older ones
- Use `^Q` (push-line) to stash a half-typed command, run something else, and get it back
- Use `LBUFFER+=` in widgets to insert text at the cursor

QUICK SUMMARY

- `bindkey -e` / `-v` — emacs / vi mode
- `bindkey KEY widget` — map a key
- `zle -N fn` — make a widget
- `$BUFFER $CURSOR` — edit the line
- `edit-command-line` — open in $EDITOR

Variables & typeset

VARIABLES

Assign with `name=value`, with **no spaces around =**. Read with `$name` or `${name}`. Variables are untyped strings by default, but `typeset` (alias `declare`) can give them attributes: integer, float, readonly, exported, upper or lower case, or unique (arrays). Variables are **global even inside functions** unless declared with `local` or `typeset`.

NOTES

- Use `${name}` when text follows: `${name}s`
- Unset variables expand to an empty string
- `typeset -p name` shows a variable's value and attributes
- `export` is `typeset -gx`

ASSIGNING & TYPESET

```
name="Ada"                    # no spaces around =
echo $name ${name}s           # Ada Adas
typeset -i count=5            # integer: assignments are math
count+=2; echo $count         # 7
count="3 * 4"; echo $count    # 12
typeset -F 2 price=3.14159    # float, 2 decimal places
echo $price                   # 3.14
typeset -r PI=3.14            # readonly (same as readonly PI=3.14)
PI=3                          # zsh: read-only variable: PI
export EDITOR=nvim            # visible to child processes
typeset -u up="shout"; echo $up   # SHOUT
typeset -l lo="QUIET"; echo $lo   # quiet
typeset -Z 3 n=7; echo $n     # 007   (zero-pad)
typeset -p name               # typeset name=Ada
f() { local x=1; y=2 }        # x is local, y leaks out as a global
```

CODE EXPLAINED

- → `typeset -i` — the right side of `=` and `+=` is evaluated as arithmetic
- → `typeset -F N` — float shown with N decimals; `-E` uses scientific notation
- → `typeset -r` — later assignments and `unset` fail
- → `typeset -u` / `-l` — case is converted on every assignment
- → `local` — same as `typeset` inside a function

TYPESET FLAGS

FLAG	ATTRIBUTE
`-i`	integer
`-F / -E`	float (fixed / exponent)
`-r`	readonly
`-x`	export
`-u / -l`	upper / lower case
`-Z N / -L N / -R N`	zero-pad, left, right justify
`-U`	unique elements (arrays)

DECLARING COMMANDS

COMMAND	SAME AS
`local x`	`typeset` x in a function
`export X=1`	`typeset -gx X=1`
`readonly X=1`	`typeset -r X=1`
`integer n`	`typeset -i n`
`float f`	`typeset -F f`

SCOPE

CODE	RESULT
`x=1 (in function)`	global
`local x=1`	this call + callees
`typeset -g x=1`	global from a function
`setopt WARN_CREATE_GLOBAL`	warns on accidental globals
`(( ${+x} ))`	true if x is set

⚠ COMMON MISTAKES

- Writing `name = value` : zsh runs a command called `name` with arguments `=` and `value`
- Forgetting `local` in functions: loop variables like `i` silently overwrite globals
- Expecting `typeset -i n=3.9` to round: it truncates to `3`
- Using `$name_suffix` when you meant `${name}_suffix` : zsh reads the whole name

✓ TIPS & BEST PRACTICES

- Use `typeset -p var` to debug: it shows type flags as well as the value
- Use `typeset -U path` to remove duplicate PATH entries automatically
- Use `local` for every function variable, and turn on `WARN_CREATE_GLOBAL` to catch slips
- Test whether a variable is set with `(( ${+var} ))` , which works even if the value is empty

QUICK SUMMARY

- `name=value` — no spaces
- `typeset -i/-F` — integer / float
- `typeset -r` / `readonly` — constant
- `export` — pass to child processes
- `local` — function scope

Special Parameters

PARAMETERS ZSH SETS FOR YOU

Zsh fills in many parameters automatically: script and function arguments, exit status, process IDs and session info. Two zsh extras matter most: `$argv`, a real **array** of the arguments, and **tied pairs** like `$path` / `$PATH`, where an array and a colon-separated string always stay in sync. Edit `$path` as an array and `$PATH` updates itself.

NOTES

- Inside a function, `$1`, `$#`, `$@` refer to the **function's** arguments
- `"$@"` keeps each argument separate; `"$*"` joins them into one word
- `$?` and `$status` are the same
- Use `${10}` (braces) for arguments past 9

ARGUMENTS & STATUS

```
# ./greet.zsh Ada "Grace Hopper" Linus
echo $0                      # ./greet.zsh
echo $1 / $2                 # Ada / Grace Hopper
echo $#                      # 3
echo $argv[-1]               # Linus   (argv is an array)
for a in "$@"; do print -r -- "<$a>"; done   # <Ada> <Grace Hopper> <Linus>
print -r -- "$*"             # Ada Grace Hopper Linus  (one word)
shift; echo $1               # Grace Hopper
false; echo $?               # 1
sleep 30 &
echo $!                      # 4182   (PID of last background job)
echo $$                      # 4170   (PID of this shell)
print -l $path               # one PATH entry per line
path=(~/bin $path)           # prepend ~/bin to PATH
echo $RANDOM $SECONDS        # 18342 12  (random 0-32767, secs since start)
```

CODE EXPLAINED

→ `$argv[-1]` — negative indexes count from the end

→ `"$@"` — the safe way to pass arguments on to another command

→ `shift` — drops `$1`; `shift 2` drops two

→ `path=(~/bin $path)` — array edit; `$PATH` is rewritten to match

ARGUMENTS

NAME	VALUE
`$0`	script or function name
`$1 … $9, ${10}`	positional arguments
`$#`	argument count
`$@ / $argv`	all arguments (array)
`$*`	all, joined in quotes
`${@[2,-1]}`	argument 2 to the end

STATUS & SESSION

NAME	VALUE
`$? / $status`	last exit code
`$$ $PPID`	shell PID, parent PID
`$!`	last background PID
`$PWD $OLDPWD`	current / previous dir
`$USERNAME $HOST`	user, hostname
`$LINENO`	current line number

TIED ARRAYS

ARRAY	STRING
`path`	PATH
`fpath`	FPATH (function path)
`cdpath`	CDPATH
`manpath`	MANPATH
`typeset -T A a`	tie your own pair

⚠ COMMON MISTAKES

- Writing `$10`: it means `${1}0`. Use `${10}`
- Passing `$*` or unquoted `$@` on: empty arguments are dropped. Use `"$@"`
- Reading `$?` after an `echo`: it now holds echo's status. Save it first: `rc=$?`
- Appending to PATH with `PATH=$PATH:~/bin` in quotes: `~` doesn't expand there. Use `path+=(~/bin)`

✓ TIPS & BEST PRACTICES

- Use `typeset -U path` so repeated `.zshrc` loads don't duplicate entries
- Use `$argv` when you want array features like `$argv[2,-1]`
- `$SECONDS` makes quick timers: `start=$SECONDS; …; echo $((SECONDS-start))`
- `zmodload zsh/datetime` gives `$EPOCHSECONDS` and `strftime`

QUICK SUMMARY

- `$1 $# "$@"` — arguments
- `$?` — last exit code
- `$$ / $!` — this shell / last job PID
- `$path ↔ $PATH` — tied array and string
- `shift` — drop the first argument

Parameter Expansion

PARAMETER EXPANSION

`${...}` does much more than read a variable. It can supply **defaults** when a value is missing, **trim** prefixes and suffixes by pattern, **replace** text, and take **substrings**. Patterns use glob syntax (*, ?, `[...]`), not regex. One # or % removes the shortest match and doubling it removes the longest. None of these change the variable except `:=`.

NOTES

- `#` trims from the front, `%` from the back (# is left of % on the keyboard)
- The colon in `:-` `:=` `:?` `:+` means "unset **or empty**"; without it, only unset
- Offsets in `${v:o:l}` start at 0; subscripts `$v[1,3]` start at 1
- Modifiers like `:t` and `:h` work too (Sheet 14)

TRIM, REPLACE & DEFAULT

```
file=/home/ada/report.final.pdf
echo ${#file}                 # 26
echo ${file##*/}              # report.final.pdf   (longest */ from front)
echo ${file%.*}               # /home/ada/report.final  (shortest .* from back)
echo ${file%%.*}              # /home/ada/report
echo ${file/ada/bob}          # /home/bob/report.final.pdf
echo ${file//a/A}             # /home/AdA/report.finAl.pdf  (all)
echo ${file:1:4}              # home   (offset 1, length 4)
echo ${file: -3}              # pdf    (space before -)

unset name
echo ${name:-guest}           # guest  (name still unset)
echo ${name:=guest}           # guest  (name is now "guest")
echo ${name:+"Hi $name"}      # Hi guest
: ${API_KEY:?must be set}     # zsh: API_KEY: must be set   (exits script)
```

CODE EXPLAINED

- → `##*/` — removes everything through the last `/`, like `basename`
- → `%.*` — removes the last extension; `%%.*` removes all of them
- → `//` — replaces every match; `/` only the first
- → `:` — the no-op command; with `:?` it validates required variables

DEFAULTS

SYNTAX	RESULT
`${v:-x}`	x if v unset/empty
`${v-x}`	x only if v unset
`${v:=x}`	also assigns x to v
`${v:?msg}`	error + exit if unset/empty
`${v:+x}`	x only if v is set
`${+v}`	1 if set, else 0

TRIM & REPLACE

SYNTAX	EFFECT
`${v#pat}`	drop shortest prefix
`${v##pat}`	drop longest prefix
`${v%pat}`	drop shortest suffix
`${v%%pat}`	drop longest suffix
`${v/a/b}` `${v//a/b}`	replace first / all
`${v/#a/b}` `${v/%a/b}`	replace at start / end

LENGTH & SUBSTRINGS

SYNTAX	RESULT
`${#v}`	length
`${v:2}`	from offset 2
`${v:2:3}`	3 chars from offset 2
`${v: -3}`	last 3 chars
`$v[1,3]`	chars 1–3 (1-based)
`$v[-1]`	last char

⚠ COMMON MISTAKES

- Writing `${v:-3}` for the last 3 characters: that's the default operator. Use `${v: -3}` or `$v[-3,-1]`
- Using regex in patterns: `${v%[0-9]+}` matches literally. Use glob syntax, or `##` with `EXTENDED_GLOB`
- Mixing up `#` and `%`: `#` trims the front, `%` the back

✓ TIPS & BEST PRACTICES

- Validate required inputs at the top of scripts with `: ${VAR:?message}`
- Get basename/dirname without subprocesses: `${f##*/}` and `${f%/*}`, or `$f:t` and `$f:h`
- Use `${v:-}` with `NO_UNSET` to read an optional variable without an error
- Expansions nest: `${${file##*/}%.*}` gives the name without its folder or extension

QUICK SUMMARY

- `${v:-x}` — default value
- `${v#p}` / `${v%p}` — trim front / back
- `${v//a/b}` — replace all
- `${v:o:l}` — substring
- `${#v}` — length

Parameter Expansion Flags

EXPANSION FLAGS

Flags are letters in parentheses right after `${`: `${(U)name}`. They transform the value: change case, **split** a string into an array, **join** an array into a string, sort, remove duplicates, quote, or look up a variable by name. Flags combine (`${(Ou)arr}`) and expansions **nest**, so the inner result feeds the outer one: `${(U)${file%.*}}`.

NOTES

- Flags taking an argument use any delimiter pair: `(s:,:)`, `(s/,/)`, `(j|-|)`
- `(f)` = `(ps:\n:)` split on newlines; `(F)` joins with newlines
- Sorting is lexical; add `n` for numeric: `(on)`
- Full list: `man zshexpn`, section *Parameter Expansion Flags*

FLAGS IN ACTION

```
s="hello world"
echo ${(U)s}                  # HELLO WORLD
echo ${(C)s}                  # Hello World
csv="b,a,c,a"
parts=${(s:,:)csv}            # split on commas → (b a c a)
echo ${#parts}                # 4
echo ${(j:-:)parts}           # b-a-c-a
echo ${(o)parts}              # a a b c   (sort ascending)
echo ${(Ou)parts}             # c b a     (descending + unique)
nums=(10 9 100); echo ${(on)nums}    # 9 10 100  (numeric sort)
lines=(${(f)"$(git branch)"}) # one element per output line
var=HOME; echo ${(P)var}      # /home/ada   (indirect lookup)
n=42; echo ${(l:5::0:)n}      # 00042   (left-pad to 5 with 0)
echo ${(U)${s%% *}}           # HELLO   (nested)
cmd='git commit -m "fix bug"'
print -l ${(z)cmd}            # git / commit / -m / "fix bug"  (shell words)
```

CODE EXPLAINED

- → `(s:,:)` — split; wrap in `( )` to store the pieces as an array
- → `(j:-:)` — join array elements with `-`
- → `(f)` — the standard way to read command output line by line
- → `(P)` — treats the value as a variable **name** and expands that
- → `(z)` — splits like the shell would, keeping quoted words together

CASE & FORMAT

FLAG	EFFECT
`(U)` `(L)`	upper / lower case
`(C)`	capitalize words
`(q)` `(qq)`	backslash / single-quote
`(Q)`	remove one level of quoting
`(l:n::c:)`	left-pad to n with c
`(r:n::c:)`	right-pad to n with c

SPLIT & JOIN

FLAG	EFFECT
`(s:x:)`	split on x
`(j:x:)`	join with x
`(f)`	split on newlines
`(F)`	join with newlines
`(z)`	split into shell words
`(@)`	keep elements in quotes

SORT & SELECT

FLAG	EFFECT
`(o)` `(O)`	sort ascending / descending
`(on)` `(oi)`	numeric / case-insensitive
`(u)`	unique, keep first
`(k)` `(v)` `(kv)`	assoc keys / values / both
`(P)`	indirect: value is a name
`(M)`	keep matches of `:#pat`

⚠ COMMON MISTAKES

- Writing `parts=${(s:,:)csv}` without `( )`: you get a string, not an array
- Forgetting quotes with `(f)`: `${(f)$(cmd)}` splits on spaces first. Use `${(f)"$(cmd)"}`
- Sorting numbers with `(o)`: `10` comes before `9`. Use `(on)`
- Putting the flag after the name, `${name(U)}`: flags go right after `${`

✓ TIPS & BEST PRACTICES

- Chain flags in one set of parentheses: `${(Lou)words}`
- Read a file into lines with `lines=(${(f)"$(<file.txt)"})`; `$(<file)` needs no `cat`
- Use `(P)` instead of `eval` for indirect variables
- Use `(q)` before building a command for `eval` to keep it safe

QUICK SUMMARY

- `(U)` `(L)` `(C)` — case
- `(s:,:)` / `(j:,:)` — split / join
- `(f)` — lines of output
- `(o)` `(O)` `(u)` — sort / unique
- `(P)` — indirect lookup

Arrays

ARRAYS

Arrays hold an ordered list of strings. Create one with `name=(a b c)`. Zsh arrays are **1-based**: `$arr[1]` is the first element, and `$arr` alone expands to **all** elements (in bash it's only the first). Negative indexes count from the end, `$arr[2,4]` takes a slice, and subscript flags like `(i)` and `(r)` search the array without a loop.

NOTES

- `$arr` (unquoted) skips empty elements; `"${arr[@]}"` keeps them
- `$#arr` and `${#arr}` both give the length
- `setopt KSH_ARRAYS` switches to bash-style 0-based indexing
- Arrays can't be nested or exported

WORKING WITH ARRAYS

```
fruits=(apple banana cherry)
echo $fruits[1]               # apple       (1-based!)
echo $fruits[-1]              # cherry
echo $fruits[2,3]             # banana cherry
echo ${#fruits}               # 3
fruits+=(date)                # append
fruits[2]=blueberry           # replace one element
fruits[2,3]=()                # delete elements 2-3
echo $fruits                  # apple date
echo ${fruits[(i)date]}       # 2   (index of first match)
echo ${fruits[(r)a*]}         # apple   (first value matching a*)
echo ${fruits:#a*}            # date    (remove values matching a*)
echo ${(M)fruits:#a*}         # apple   (keep only matches)
(( $fruits[(Ie)kiwi] )) || echo "no kiwi"   # (I) gives 0 if missing
nums=({1..5})                 # 1 2 3 4 5
for f in $fruits; do echo "- $f"; done
```

CODE EXPLAINED

- → `arr[i,j]=()` — removes a range; `arr[i]=()` removes one element
- → `(i)` — index of first match; returns **length + 1** if not found
- → `(I)` — index of last match; returns **0** if not found, so it works as a test
- → `(e)` — exact match, so `*` in the search isn't a pattern
- → `:#pat` — filters elements; `(M)` flips it to keep matches

ACCESS

SYNTAX	RESULT
`$a[1]`	first element
`$a[-1]`	last element
`$a[2,4]`	elements 2–4
`$a[2,-1]`	all but the first
`${#a}`	number of elements
`"${a[@]}"`	all, keeping empties

MODIFY

SYNTAX	EFFECT
`a+=(x y)`	append
`a=(x $a)`	prepend
`a[3]=x`	set element 3
`a[3]=()`	delete element 3
`shift a`	drop first element
`a=()`	empty the array

SEARCH & SET OPS

SYNTAX	RESULT
`${a[(i)x]}`	first index of x
`${a[(Ie)x]}`	last index, 0 if none
`${a[(r)pat]}`	first value matching
`${a:#pat}`	remove matches
`${a:|b}`	in a but not in b
`${a:*b}`	in both a and b

⚠ COMMON MISTAKES

- Using `$arr[0]` : in zsh that's empty. The first element is `$arr[1]`
- Writing `arr=(a, b, c)` : commas become part of the values. Separate with spaces
- Testing membership with `(i)` and `!= 0` : it never returns 0. Use `(Ie)` or compare with `${#a}+1`
- Copying with `b=$a` : you get one joined string. Use `b=("${a[@]}")`

✓ TIPS & BEST PRACTICES

- Use `"${arr[@]}"` when elements may be empty or must stay separate
- Set operations need no loops: `${a:|b}` (difference) and `${a:*b}` (intersection)
- Remove duplicates with `typeset -U arr` or `${(u)arr}`
- Use `print -l $arr` to see each element on its own line

QUICK SUMMARY

- `a=(x y z)` — create
- `$a[1]` / `$a[-1]` — first / last (1-based)
- `$a[2,3]` — slice
- `a+=(x)` — append
- `${a[(Ie)x]}` — find (0 if missing)

Associative Arrays

ASSOCIATIVE ARRAYS

An associative array maps **string keys to values**, like a dictionary or hash map. It must be declared with `typeset -A` (or `local -A`) before use. Assign pairs with `h=(k1 v1 k2 v2)` or `h[key]=value`, and read with `$h[key]`. Expansion flags list the keys `(k)`, values `(v)`, or both `(kv)`. Key order is **not** preserved, so sort with `(ok)` when order matters.

NOTES

- Without `typeset -A`, `h[name]=x` is treated as a numeric index and fails
- Values are strings; there are no nested hashes
- `$h` alone expands to all values
- Quote keys with spaces or special chars: `$h["my key"]`

USING A HASH

```
typeset -A user                # declare first
user=(name Ada lang zsh)       # key value key value …
user[city]=London              # add or update a key
echo $user[name]               # Ada
echo ${(k)user}                # name lang city   (order not guaranteed)
echo ${(v)user}                # Ada zsh London
echo ${#user}                  # 3
(( ${+user[city]} )) && echo "has city"     # key exists?
unset 'user[city]'             # delete a key (quote it)
for k v in ${(kv)user}; do print "$k=$v"; done
for k in ${(ok)user}; do print "$k → $user[$k]"; done   # sorted keys
typeset -A codes=( [red]=31 [green]=32 )   # [key]=value form
echo ${codes[(r)3*]}           # 31 or 32   (first value matching)
echo ${codes[(I)g*]}           # green      (key matching pattern)
```

CODE EXPLAINED

- → `${+h[key]}` — 1 if the key exists, even when its value is empty
- → `unset 'h[key]'` — quotes stop `[key]` being read as a glob
- → `for k v in` — zsh can take two loop variables at a time
- → `(r)` / `(I)` — search values / keys by pattern

OPERATIONS

SYNTAX	EFFECT
`typeset -A h`	declare
`h=(k v k v)`	replace all pairs
`h[k]=v`	set one key
`h+=(k v)`	add pairs
`unset 'h[k]'`	delete a key
`h=()`	clear

READING

SYNTAX	RESULT
`$h[k]`	value for k
`${(k)h}`	all keys
`${(v)h}`	all values
`${(kv)h}`	keys and values
`${(ok)h}`	keys, sorted
`${#h}`	number of pairs

TESTING & SEARCHING

SYNTAX	RESULT
`(( ${+h[k]} ))`	true if key exists
`${h[(r)pat]}`	first value matching
`${h[(R)pat]}`	all values matching
`${h[(i)pat]}`	first key matching
`${(k)h[(R)pat]}`	keys whose value matches
`typeset -p h`	dump the whole hash

⚠ COMMON MISTAKES

- Skipping `typeset -A`: assigning `h[key]=v` then fails with a subscript error
- Expecting keys in insertion order: zsh hashes are unordered. Use `(ok)` or keep a separate key array
- Testing `[[ -n $h[k] ]]` for existence: it's false for keys with empty values. Use `${+h[k]}`
- Writing `local h` inside a function and then `h[k]=v`: declare it as `local -A h`

✓ TIPS & BEST PRACTICES

- Use `typeset -p h` to inspect a hash quickly while debugging
- Copy a hash with `typeset -A b=("${(@kv)a}")`
- Use a hash as a set: `seen[$x]=1`, then check `(( ${+seen[$x]} ))`
- Build a lookup table at the top of a script instead of a long `case`

QUICK SUMMARY

- `typeset -A h` — declare
- `h[k]=v` / `$h[k]` — set / get
- `${(k)h}` `${(v)h}` — keys / values
- `${+h[k]}` — key exists?
- `for k v in ${(kv)h}` — iterate pairs

Globbing Basics

FILENAME GENERATION

Globbing turns patterns into lists of matching filenames before a command runs. Zsh supports the standard *, ? and `[...]`, and adds **recursive `/`**, numeric ranges **`<1–10>`, and alternatives `(a|b)`. With `setopt EXTENDED_GLOB` you also get** negation **`^`**, exclusion **`~`, and repetition `#`. By default a pattern that matches nothing is an** error** (`no matches found`), not a literal string.

NOTES

- Globs skip dotfiles unless the pattern starts with `.` or you use `(D)` / `GLOB_DOTS`
- Brace expansion `{a,b}` isn't globbing: it builds strings whether files exist or not
- Quote patterns meant for other programs: `find . -name '*.py'`
- `noglob cmd` turns globbing off for one command

PATTERNS

```
ls *.txt                    # every .txt here
ls file?.log                # file1.log, fileA.log
ls [a-c]*.md                # starting with a, b or c
ls [^0-9]*                  # not starting with a digit
ls **/*.py                  # recursive: .py files at any depth
ls ***/*.py                 # recursive, following symlinked dirs
ls log<1-10>.txt            # log1.txt … log10.txt (that exist)
ls img<->.png               # img + any number
ls (README|LICENSE)*        # alternatives
echo {a,b,c}.txt            # a.txt b.txt c.txt   (no files needed)
echo {1..5} {01..03}        # 1 2 3 4 5 01 02 03
setopt EXTENDED_GLOB
ls ^*.bak                   # everything except *.bak
ls *.log~error*             # .log files, excluding error*
ls *.(jpg|png)(N)           # (N): empty if no match, no error
```

CODE EXPLAINED

- → **`/` — zero or more directories, so** `/*.py` includes `./x.py`
- → `<1-10>` — matches numbers by **value**, so `log07.txt` matches too
- → `<->` — any non-negative integer
- → `(N)` — the NULL_GLOB qualifier for one pattern (Sheet 14)

BASIC WILDCARDS

PATTERN	MATCHES
`*`	any string
`?`	one character
`[abc]` `[a-z]`	one char from set
`[^abc]` `[!abc]`	one char not in set
`[[:digit:]]`	character class
`(a|b)`	a or b

ZSH ADDITIONS

PATTERN	MATCHES
`**/`	any depth of dirs
`***/`	same, follows symlinks
`<1-10>`	numbers 1 to 10
`<5-> <-5>`	≥ 5 / ≤ 5
`{a,b} {1..9}`	brace expansion
`{a-e}`	needs BRACE_CCL

EXTENDED_GLOB

PATTERN	MATCHES
`^pat`	anything except pat
`a~b`	a, but not b
`x#`	zero or more x
`x##`	one or more x
`(#i)pat`	case-insensitive
`(#b)(*)`	capture into `$match`

⚠ COMMON MISTAKES

- `curl https://x.com/?q=1` fails with `no matches found`: `?` is a glob. Quote URLs
- `git add *.js` stops the command when nothing matches. Add `(N)` or quote it
- Using `ls` **for recursion: you need** `/`
- Expecting `^` and `~` to work without `setopt EXTENDED_GLOB`

✓ TIPS & BEST PRACTICES

- Use `**/*.ext(.)` instead of `find . -name` for quick searches
- Add `alias curl='noglob curl'` (and wget, git) to stop URL glob errors
- Preview a destructive glob first: `print -l **/*.tmp` before `rm`
- Use `(#i)` for one case-insensitive pattern without changing options

QUICK SUMMARY

- `* ? [..]` — standard wildcards
- `**/` — recursive
- `<1-10>` — numeric range
- `^ ~` — not / except (EXTENDED_GLOB)
- `(N)` — no error on no match

Glob Qualifiers & Modifiers

(.) (/) (@) · (om[1]) newest · (Lk+100) size · (mh-1) mtime

QUALIFIERS & MODIFIERS

A **glob qualifier** is a parenthesized suffix that filters or sorts the matches: `*(.)` keeps only regular files, `*(om[1])` picks the newest. Qualifiers can test type, size, age, permissions and owner, and they combine (`*(.Lm+10om)`). **Modifiers** like `:h` and `:t` edit a path; they work on variables (`$f:t`), history (`!$:h`), and inside qualifiers (`*(:r)`).

NOTES

- Combine qualifiers in one set: `(.Lk+500om[1,5])`
- `^` negates the next qualifier: `*(^/)` = not directories
- Time units: `s` `m` `h` `d` (default) `w` `M`; size units: `k` `m`
- `+n` means more than n, `-n` less than n

FILTER, SORT & EDIT PATHS

```
ls *(.)                     # regular files only
ls -d *(/)                  # directories only
ls *(@)                     # symlinks only
ls *(*)                     # executable files
ls -l *(.om[1])             # newest file
ls *(.om[1,3])              # 3 most recently modified
ls **/*(.Lm+100)            # files over 100 MB, any depth
ls *(.mh-1)                 # modified in the last hour
rm *.log(.md+30)            # .log files older than 30 days
ls *(D)                     # include dotfiles
print -l **/*(/^F)          # empty directories
rm *.tmp(N)                 # no error if nothing matches
f=~/src/app/main.test.js
echo $f:h                   # /home/ada/src/app
echo $f:t $f:e $f:t:r       # main.test.js js main.test
echo *.md(:r)               # file names without .md
```

CODE EXPLAINED

- → `om` — sort by modification time, newest first; `[1]` takes the first
- → `Lm+100` — size (`L`) in megabytes greater than 100
- → `mh-1` — modified less than 1 hour ago; `md+30` — more than 30 days
- → `^F` — not "full", i.e. empty directories
- → `:t:r` — modifiers chain left to right

TYPE QUALIFIERS

QUAL	SELECTS
`(.)`	regular files
`(/)`	directories
`(@)`	symlinks
`(*)`	executables
`(D)`	include dotfiles
`(N)`	empty list if no match
`(-.)`	follow links, then test

SORT, SIZE & TIME

QUAL	MEANING
`(om)` `(Om)`	newest / oldest first
`(oL)` `(OL)`	smallest / largest first
`(on)`	by name
`([1])` `([-1])`	first / last match
`(Lk+500)`	larger than 500 KB
`(mw-2)`	changed in the last 2 weeks
`(u:ada:)`	owned by user ada

MODIFIERS

MOD	ON /A/B/C.TAR.GZ
`:h`	/a/b
`:t`	c.tar.gz
`:r`	/a/b/c.tar
`:e`	gz
`:a` `:A`	absolute / resolve symlinks
`:u` `:l`	upper / lower case
`:s/a/b/`	replace first a

⚠ COMMON MISTAKES

- Writing `ls *(/)` and seeing contents: `ls` lists inside dirs. Use `ls -d *(/)` or `print -l`
- Mixing up `m` (days) and `mm` (minutes): `mm-5` is the last 5 minutes
- Running `rm` on a qualifier glob without previewing: check with `print -l` first
- Expecting qualifiers to work inside quotes: `"*(.)"` is literal

✓ TIPS & BEST PRACTICES

- Open the newest download: `open ~/Downloads/*(.om[1])`
- Find big files fast: `print -l **/*(.OL[1,10])` lists the 10 largest
- Turn off `BARE_GLOB_QUAL` only if you need literal `(...)` in filenames
- Use `:A` to resolve a script's real path: `${0:A:h}`

QUICK SUMMARY

- `*(.)` `*(/)` — files / dirs
- `(om[1])` — newest
- `(Lm+N)` `(md-N)` — size / age
- `(N)` `(D)` — null glob / dotfiles
- `:h` `:t` `:r` `:e` — path parts

Test Expressions

TESTING CONDITIONS

`[[ ... ]]` is zsh's conditional expression. It checks files, compares strings, and matches patterns or regular expressions, then sets an exit status: **0 means true**. Because `[[` is a shell keyword, variables inside it are not split or globbed, so quoting is rarely needed. The right side of == is a **pattern** unless you quote it. For numbers, `(( ))` is usually clearer (Sheet 17).

NOTES

- `[ ]` / `test` is the old POSIX command: quote every variable and use `=`
- In zsh, `=~` treats a quoted right side as a regex too (unlike bash)
- `$MATCH` = whole regex match, `$match[n]` = capture groups
- `setopt RE_MATCH_PCRE` switches `=~` to Perl-style regex

COMMON TESTS

```
[[ -f notes.txt ]] && echo "is a file"
[[ -d ~/src ]] || mkdir ~/src
[[ -z $name ]] && echo "empty"          # zero length
[[ -n $name ]] && echo "has a value"
[[ $answer == y* ]] && echo yes         # pattern: y, yes, yep …
[[ $ver == <1-9>.* ]]                   # zsh numeric glob in a test

s="2024-ok"
if [[ $s =~ '^([0-9]+)-(.*)$' ]]; then
  echo $MATCH $match[1] $match[2]       # 2024-ok 2024 ok
fi

[[ a.txt -nt b.txt ]] && echo "a is newer"
[[ $n -gt 5 && $n -lt 10 ]]             # numeric compare
(( n > 5 && n < 10 ))                   # same, arithmetic form
```

CODE EXPLAINED

→ `&&` / `||` — run the next command only on success / failure

→ `==` — glob pattern match; `=` works the same inside `[[ ]]`

→ `=~` — extended regex; groups land in `$match`

→ `-nt` — newer than (modification time); `-ot` older than

FILE TESTS

TEST	TRUE IF
`-e f`	exists
`-f f`	regular file
`-d f`	directory
`-L f`	symlink
`-s f`	size > 0
`-r / -w / -x f`	readable / writable / executable
`a -nt b`	a newer than b

STRING TESTS

TEST	TRUE IF
`-z s`	empty
`-n s`	not empty
`a == pat`	a matches pattern
`a != pat`	a doesn't match
`a < b`	sorts before (string)
`a =~ re`	regex match

NUMBERS & LOGIC

TEST	MEANING
`-eq -ne`	equal / not equal
`-lt -le`	< / ≤
`-gt -ge`	> / ≥
`&& ||`	and / or
`!`	not

⚠ COMMON MISTAKES

- Comparing numbers with `<` in `[[ ]]`: `10 < 9` is true as strings. Use `-lt` or `(( ))`
- Quoting a pattern you meant to match: `[[ $f == "*.txt" ]]` only matches a literal `*.txt`
- Leaving out the spaces: `[[$x == y]]` is a syntax error
- Using `[ $x = y ]` with an empty `$x`: the test breaks. Use `[[ ]]`

✓ TIPS & BEST PRACTICES

- Prefer `[[ ]]` in zsh scripts; use `[ ]` only for strict POSIX `sh`
- Use `(( ))` for numbers: `(( count > 0 ))` reads like math
- Store complex regexes in a variable: `re='^v([0-9]+)'; [[ $t =~ $re ]]`
- Test commands directly: `if grep -q x file` instead of `[[ $(grep …) ]]`

QUICK SUMMARY

- `[[ -f f ]]` — file checks
- `[[ -z s ]]` / `-n` — empty / not empty
- `[[ s == pat ]]` — pattern match
- `[[ s =~ re ]]` — regex, `$match`
- `(( a > b ))` — numeric compare

if, case & select

BRANCHING

`if` runs a command and branches on its **exit status**: 0 is true, anything else is false. The condition is often `[[ ]]` or `(( ))`, but any command works (`if grep -q …`). `case` matches one value against glob patterns, top to bottom, and runs the first match. Zsh adds two extra terminators: `;&` falls through to the next body and `;|` keeps testing. `select` builds a numbered menu from a list.

NOTES

- Zsh also accepts brace forms: `if [[ … ]] { … } else { … }`
- `case` patterns are globs, not regexes; `*)` is the default
- `select` puts the choice in a variable and the typed number in `$REPLY`

IF, CASE & SELECT

```
if [[ $1 == -h ]]; then
  print "usage: deploy ENV"
elif (( $# == 0 )); then
  print -u2 "no environment"; exit 1
else
  deploy "$1"
fi

case $file in
  *.jpg|*.png) echo image ;;
  *.tar.gz)    echo tarball ;&    # ;& runs the next body too
  *.zip)       echo archive ;;
  *)           echo other ;;
esac

PS3="Pick a color: "
select c in red green blue; do
  [[ -n $c ]] && { echo "You chose $c (#$REPLY)"; break }
done
```

CODE EXPLAINED

- → `elif` / `else` — optional; close with `fi`
- → `a|b)` — either pattern matches
- → `;&` — skips the next pattern test and runs its body
- → `PS3` — the prompt `select` shows
- → `$c` is empty when the input isn't a valid number

CASE TERMINATORS

END	THEN
`;;`	stop (normal)
`;&`	run next body, no test
`;|`	test the next patterns
`*)`	default branch
`(#i)yes)`	case-insensitive (EXTENDED_GLOB)
`<1-5>)`	numeric range

CONDITION FORMS

FORM	TRUE WHEN
`if cmd`	cmd exits 0
`if ! cmd`	cmd fails
`if [[ … ]]`	test passes
`if (( … ))`	math is non-zero
`a && b || c`	**not** a true if/else
`[[ … ]] && { … }`	one-line if

⚠ COMMON MISTAKES

- Using `a && b || c` as if/else: if `b` fails, `c` runs too
- Forgetting `;;` : the next pattern line becomes part of the current body or a syntax error
- Quoting `case` patterns: `"*.txt")` only matches a literal `*.txt`
- Missing `break` in `select` : the menu repeats forever

✓ TIPS & BEST PRACTICES

- Order `case` patterns from most to least specific; the first match wins
- Use `;|` to tag a value with every matching category
- Branch on commands directly: `if command -v brew >/dev/null; then …`
- Use the brace form for short one-line conditionals

QUICK SUMMARY

- `if … elif … else … fi` — branching
- `case $x in pat) … ;; esac` — matching
- `;&` / `;|` — fall through / keep testing
- `select x in …` — numbered menu

Arithmetic

MATH IN ZSH

`$(( expr ))` evaluates arithmetic and substitutes the result; `(( expr ))` evaluates it as a **command** whose status is true when the result is non-zero. Unlike bash, zsh does **floating point** math natively: if any operand is a float, the result is a float. Inside `(( ))`, variables need no $. Load `zsh/mathfunc` for `sqrt`, `floor`, `sin` and other functions.

NOTES

- Integer division truncates: `7 / 2` is `3`
- Write `2.0` (not `2`) to force float math
- `16#ff` reads a number in base 16
- `[#16]` prints the result in base 16

ARITHMETIC

```
echo $(( 7 + 3 * 2 ))          # 13
echo $(( 7 / 2 ))              # 3      (integer division)
echo $(( 7 / 2.0 ))            # 3.5    (float)
echo $(( 7 % 3 ))              # 1
echo $(( 2 ** 10 ))            # 1024
x=5
(( x++ )); echo $x             # 6      (no $ inside)
(( x += 10 )); echo $x         # 16
(( x > 10 )) && echo big       # big
echo $(( x > 10 ? 1 : 0 ))     # 1      (ternary)
echo $(( 16#ff ))              # 255
echo $(( [#16] 255 ))          # 16#FF
echo $(( [##2] 5 ))            # 101
echo $(( 10.0 / 3 ))           # 3.3333333333333335
zmodload zsh/mathfunc
echo $(( sqrt(16) ))           # 4.
printf "%.2f\n" $(( 10.0 / 3 ))   # 3.33
```

CODE EXPLAINED

- → `**` — power; `%` — remainder
- → `(( x++ ))` — returns false when `x` was 0, which matters with `ERR_EXIT`
- → `[#16]` — output base with prefix; `[##16]` without
- → `sqrt(16)` — float result prints as `4.`; wrap in `int()` for `4`
- → `printf "%.2f"` — the easiest way to round for display

OPERATORS

OP	MEANING
`+ - * /`	basic math
`% **`	remainder, power
`++ --`	increment, decrement
`+= -= *= /=`	compound assign
`== != < <= > >=`	compare
`&& || !`	logic
`& | ^ << >>`	bitwise

FORMS

SYNTAX	USE
`$(( e ))`	value of e
`(( e ))`	status: true if ≠ 0
`let 'e'`	same as ((e))
`integer n`	typed integer var
`float f`	typed float var
`$[ e ]`	old form, avoid

ZSH/MATHFUNC

FUNCTION	RETURNS
`sqrt(x)`	square root
`abs(x)`	absolute value
`floor(x) ceil(x)`	round down / up
`rint(x)`	round to nearest
`int(x) float(x)`	convert type
`log(x) exp(x)`	natural log, e^x
`sin(x) cos(x)`	trig (radians)

⚠ COMMON MISTAKES

- Expecting `$(( 1 / 3 ))` to be `0.33`: both are integers, so it's `0`. Use `1.0 / 3`
- Writing `$(( $x++ ))`: that becomes `5++`. Use `(( x++ ))`
- Using `(( count++ ))` with `ERR_EXIT` when count is 0: the status is false and the script exits
- Forgetting `zmodload zsh/mathfunc`: `sqrt` is an unknown function without it

✓ TIPS & BEST PRACTICES

- Use `(( ))` for all number comparisons instead of `-gt`
- Format floats with `printf '%.2f'` or `typeset -F 2`
- Use `(( x++ )) || true`, or `(( ++x ))`, in scripts with `ERR_EXIT`
- Leading zeros are decimal in zsh (`010` is 10) unless `OCTAL_ZEROES` is set

QUICK SUMMARY

- `$(( ))` — math result
- `(( ))` — math test
- `2.0` — any float makes float math
- `16#ff` / `[#16]` — bases
- `zmodload zsh/mathfunc` — sqrt, floor …

Loops

LOOPING

`for` runs a block once per word in a list, a range, or C-style counter. `while` repeats while a command succeeds and `until` repeats while it fails. Zsh adds `repeat N`, **short forms** that skip `do … done` for one command, and the ability to take **several variables per pass**. Lists come from globs, arrays, brace ranges, or command output split into lines with `${(f)…}`.

NOTES

- `{1..$n}` works with variables in zsh
- `{0..10..2}` adds a step: 0 2 4 6 8 10
- `while read -r line` is the safe way to read lines
- `break 2` / `continue 2` affect the outer loop

LOOP FORMS

```
for f in *.txt; do
  echo "→ $f"
done
for f in *.txt; echo $f                  # short form, one command
for x ($fruits) print $x                 # short form with ( )
for i in {1..5}; do echo $i; done
for (( i = 0; i < 3; i++ )); do echo $i; done        # 0 1 2
for k v in ${(kv)user}; do echo "$k=$v"; done       # two vars per pass
for host in ${(f)"$(< hosts.txt)"}; do ping -c1 $host; done
while read -r line; do
  echo "> $line"
done < input.txt
until curl -sf localhost:8080 >/dev/null; do sleep 2; done
repeat 3 print hi                        # hi hi hi (3 lines)
for i in {1..10}; do
  (( i % 2 )) && continue                # skip odd numbers
  (( i > 6 )) && break                   # stop after 6
  echo $i                                # 2 4 6
done
```

CODE EXPLAINED

- → `for f in *.txt` — glob expands once, before the loop starts
- → `for k v in` — takes two words per pass
- → `${(f)"$(< file)"}` — one element per line, spaces kept
- → `while read -r … done < file` — reads in the current shell, so variables persist
- → `repeat N` — runs a command N times

LOOP FORMS

SYNTAX	USE
`for x in a b c`	list of words
`for x ($arr) cmd`	zsh short form
`for ((i=0;i<n;i++))`	C-style counter
`while cmd`	while it succeeds
`until cmd`	until it succeeds
`repeat N`	fixed count

LIST SOURCES

SYNTAX	GIVES
`{1..10}`	1 to 10
`{0..20..5}`	0 5 10 15 20
`*(.)`	regular files
`$arr`	array elements
`${(f)"$(cmd)"}`	lines of output
`${(kv)h}`	key/value pairs

LOOP CONTROL

SYNTAX	EFFECT
`break`	exit the loop
`break 2`	exit 2 levels
`continue`	next iteration
`continue 2`	next outer iteration
`return`	leave the function
`Ctrl-C`	interrupt loop

⚠ COMMON MISTAKES

- Changing an array inside a loop over it: the list was expanded when the loop started
- Writing `for line in $(cat file)` : it splits on spaces as well as newlines. Use `${(f)"$(<file)"}`
- Forgetting `-r` on `read` : backslashes are eaten
- Using `for i in 1..5` : you need braces, `{1..5}`

✓ TIPS & BEST PRACTICES

- Use the short form `for f (*.log) gzip $f` for quick one-liners
- Add `(N)` to loop globs so an empty match skips the loop instead of erroring
- Use `while IFS=, read -r a b` to split CSV-like lines into fields
- Loop over lines safely with `while read -r line; do …; done < <(cmd)`

QUICK SUMMARY

- `for x in list; do …; done` — iterate
- `for ((;;))` — counter loop
- `while` / `until` — conditional loops
- `repeat N` — fixed count
- `break` / `continue` — loop control

Functions

FUNCTIONS

A function is a named group of commands that runs in the **current shell**, so it can change variables and the working directory. Arguments arrive as $1, $@, $#. The `return` value is only an **exit status** (0–255): to give back data, **print** it and capture it with `$( )`. Zsh adds anonymous functions, which run immediately, and **autoloading**, which loads functions from files in `$fpath` on first use.

NOTES

- Variables are global unless declared `local`
- `$0` inside a function is the function's name
- Functions take priority over commands of the same name
- `command ls` / `builtin cd` skip a function wrapper

DEFINING & CALLING

```
greet() {
  local name=${1:-world}      # first arg, default "world"
  print "Hello, $name"
}
greet Ada                     # Hello, Ada
function add { print $(( $1 + $2 )) }
sum=$(add 2 3); echo $sum     # 5   (capture printed output)
is_even() { (( $1 % 2 == 0 )) }   # last status is the result
is_even 4 && echo even        # even
fail() { return 3 }; fail; echo $?        # 3
show() { echo "$0 got $#: $@" }; show a b # show got 2: a b
() { echo "anon got $1" } hi              # anon got hi  (runs now)
mkcd() { mkdir -p -- "$1" && cd -- "$1" }   # changes caller's dir

fpath=(~/.zsh/functions $fpath)   # file ~/.zsh/functions/backup
autoload -Uz backup           # loaded on first call
functions greet               # print the definition
unfunction greet              # delete it
```

CODE EXPLAINED

- → `local` — keeps the variable inside this call (and functions it calls)
- → `$( … )` — runs in a subshell, so `cd` or variable changes inside are lost
- → `return N` — sets `$?`; without it, the last command's status is returned
- → `() { … } args` — anonymous function: good for a private scope in `.zshrc`
- → `autoload -Uz` — `-U` skips aliases, `-z` uses zsh-style loading

DEFINING

SYNTAX	NOTE
`name() { … }`	POSIX style
`function name { … }`	ksh style
`() { … } args`	anonymous, runs at once
`autoload -Uz f`	load from $fpath
`functions f`	show definition
`unfunction f`	remove

INPUT & OUTPUT

SYNTAX	MEANING
`$1 $2 … $#`	arguments, count
`"$@"`	all args, separately
`local x`	function-scoped var
`print …`	return data (capture)
`return N`	exit status 0–255
`REPLY=x`	convention: result var

FINDING COMMANDS

COMMAND	SHOWS
`whence -v f`	what f is
`which f`	definition or path
`type -a f`	every match
`$funcstack`	current call stack
`print -l $fpath`	autoload dirs
`command f`	skip functions

⚠ COMMON MISTAKES

- Using `return "text"`: return only takes a number. Print the text and capture it
- Calling `cd` inside `$(fn)`: the subshell changes directory, not your shell
- Forgetting `local`: helper variables leak and overwrite globals
- Naming a function after a command (`ls() { ls -G }`): infinite recursion. Use `command ls -G`

✓ TIPS & BEST PRACTICES

- Return data through `REPLY` to avoid a subshell: `fn; echo $REPLY`
- Keep big functions as files in `~/.zsh/functions` and autoload them
- Wrap `.zshrc` setup in `() { local … }` so temporary variables don't stay around
- Start reusable functions with `emulate -L zsh`

QUICK SUMMARY

- `f() { … }` — define
- `$1 "$@"` — arguments
- `local` — scope variables
- `print` + `$( )` — return data
- `autoload -Uz` — lazy-load from $fpath

Scripts, Errors & Debugging

ROBUST SCRIPTS

A zsh script is a text file starting with `#!/usr/bin/env zsh` and marked executable. For safer scripts, turn on **strict options** so errors stop execution, clean up with `trap` or an **`always` block** (zsh's try/finally), and parse flags with the `zparseopts` builtin. When something goes wrong, XTRACE prints every command before it runs.

NOTES

- `ERR_EXIT` = `set -e`, `NO_UNSET` = `set -u`
- `ERR_RETURN` returns from the function instead of exiting
- Check syntax without running: `zsh -n script.zsh`
- `${0:A:h}` is the script's own directory

SCRIPT SKELETON

```
#!/usr/bin/env zsh
setopt NO_UNSET PIPE_FAIL                # strict mode

zparseopts -D -E -F -- h=help v=verbose o:=out || exit 1
(( $#help )) && { print "usage: ${0:t} [-v] [-o FILE] ARGS"; exit 0 }
outfile=${out[-1]:-out.txt}              # value after -o, or default

tmp=$(mktemp)
trap 'rm -f $tmp' EXIT                   # cleanup on any exit
TRAPINT() { print -u2 "interrupted"; return 130 }

main() {
  setopt LOCAL_OPTIONS ERR_RETURN        # stop at the first failure
  build_report > $tmp
  mv $tmp $outfile
}
{ main "$@" } always { print -u2 "done (status $?)" }
```

CODE EXPLAINED

- → `zparseopts -D` — removes parsed flags from `$@`; `-E` allows unknown args; `-F` fails on bad flags
- → `o:=out` — `-o` takes a value; `$out` becomes `(-o FILE)`
- → `trap … EXIT` — runs when the script exits, for any reason
- → `ERR_RETURN` — makes `main` return at its first failing command
- → `{ } always { }` — like `finally`: runs after errors and returns, but **not** after `exit`

STRICT OPTIONS

OPTION	EFFECT
`ERR_EXIT`	exit on failure
`ERR_RETURN`	return from function
`NO_UNSET`	unset var = error
`PIPE_FAIL`	failed pipe part = failure
`WARN_CREATE_GLOBAL`	flag accidental globals
`LOCAL_TRAPS`	traps local to function

TRAPS

SYNTAX	RUNS ON
`trap 'c' EXIT`	shell or function exit
`trap 'c' INT TERM`	Ctrl-C, kill
`trap 'c' ZERR`	any failing command
`TRAPEXIT() {…}`	function form
`trap '' INT`	ignore signal
`trap - INT`	reset to default

DEBUGGING

TOOL	EFFECT
`zsh -n f`	syntax check only
`zsh -x f`	trace every command
`setopt XTRACE`	trace from here on
`PS4='+%N:%i> '`	trace prefix (file:line)
`functions -t f`	trace one function
`typeset -p v`	inspect a variable

⚠ COMMON MISTAKES

- Relying on `ERR_EXIT` inside `if`, `&&` or `||`: failures there are ignored by design
- Counting on `always` with `ERR_EXIT` or `exit`: the shell exits first. Use a `trap … EXIT` for cleanup that must run
- Reading `$out` as the value: zparseopts stores `(-o FILE)`. Use `$out[-1]`

✓ TIPS & BEST PRACTICES

- Use `print -u2` for messages so stdout stays clean for data
- Find a script's folder with `${0:A:h}` to load files next to it
- Exit with clear codes: `exit 2` for usage errors, `exit 1` for failures

QUICK SUMMARY

- `#!/usr/bin/env zsh` — shebang
- `ERR_EXIT NO_UNSET PIPE_FAIL` — strict mode
- `zparseopts` — flag parsing
- `trap … EXIT` / `always` — cleanup
- `zsh -x` / `XTRACE` — trace

Jobs & Process Substitution

JOBS & SUBSTITUTION

Ending a command with & runs it as a **background job**. `jobs`, `fg`, `bg` and Ctrl-Z move jobs between the foreground, background and stopped states. **Command substitution** `$(cmd)` captures output as text, while **process substitution** turns a command's output into something that looks like a file: `<(cmd)` (a pipe path), `>(cmd)` (a pipe you write into) and zsh's `=(cmd)` (a real temp file).

NOTES

- Refer to jobs as `%1`, `%+` (current), `%-` (previous), `%vim` (by name)
- Zsh warns about running jobs on exit; exit again to confirm
- `=(cmd)` works with programs that need to seek, like editors
- `$(<file)` reads a file without running `cat`

JOBS & SUBSTITUTION

```
sleep 60 &                      # [1] 4182
jobs -l                         # [1]  + 4182 running    sleep 60
fg %1                           # bring job 1 to the foreground
# press Ctrl-Z to stop it, then:
bg                              # continue it in the background
kill %1                         # stop job 1
server.sh &!                    # start already disowned (no job entry)
disown %2                       # stop tracking an existing job
wait                            # wait for all background jobs

today=$(date +%F)               # capture output as text
diff <(sort a.txt) <(sort b.txt)    # compare sorted output
vim =(curl -s example.com)      # open output as a temp file
tar cf >(gzip > site.tgz) site/     # write through a command
coproc bc -l                    # two-way pipe to bc
print -p "2^10"; read -p ans; echo $ans   # 1024
```

CODE EXPLAINED

→ `&!` (or `&|`) — background and disown at once; survives closing the terminal

→ `<(cmd)` — gives a path like `/proc/self/fd/11` to read from

→ `=(cmd)` — writes output to a temp file and passes its name; deleted afterwards

→ `coproc` — `print -p` writes to it and `read -p` reads from it

JOB CONTROL

SYNTAX	EFFECT
`cmd &`	run in background
`Ctrl-Z`	stop foreground job
`jobs -l`	list jobs with PIDs
`fg %n`	resume in foreground
`bg %n`	resume in background
`kill %n`	terminate a job

DETACHING

SYNTAX	EFFECT
`cmd &!`	background + disown
`disown %n`	forget a job
`nohup cmd &`	ignore hangup, log to nohup.out
`setopt NO_HUP`	don't kill jobs on exit
`wait %n`	wait for a job
`wait`	wait for all

SUBSTITUTION

SYNTAX	GIVES
`$(cmd)`	output as text
`$(<file)`	file contents
`<(cmd)`	readable pipe path
`>(cmd)`	writable pipe path
`=(cmd)`	temp file path
`` `cmd` ``	old form of $( )

⚠ COMMON MISTAKES

- Closing the terminal with jobs running: they get killed. Use `&!` or `disown`
- Using `<(cmd)` with a program that seeks or reopens the file: it fails. Use `=(cmd)`
- Expecting `$(cmd)` to keep trailing newlines: they're stripped
- Using backticks in new code: they're hard to nest. Use `$( )`

✓ TIPS & BEST PRACTICES

- Diff two command outputs with `diff <(cmd1) <(cmd2)`
- Run tasks in parallel with `&`, then `wait` for all of them
- Use `%name` to refer to jobs by command: `fg %vim`
- Use `kill -STOP %1` / `kill -CONT %1` to pause and resume a background job

QUICK SUMMARY

- `&` / `fg` / `bg` — job control
- `&!` — start detached
- `$(cmd)` — capture output
- `<(cmd)` / `=(cmd)` — output as a file
- `wait` — wait for jobs

Aliases & Directory Navigation

SHORTCUTS FOR TYPING & MOVING

An **alias** replaces a word with text before the command runs. Zsh has three kinds: regular aliases (only in command position), **global** aliases with –g (anywhere on the line), and **suffix** aliases with –s (open a file by typing its name). For navigation, zsh can `cd` when you type a bare directory name, keep a numbered **directory stack**, and give folders short **named directory** names like `~proj`.

NOTES

- Aliases aren't expanded in scripts unless `ALIASES` is set and they're defined there
- Escape an alias with `\ls` or `command ls`
- Named dirs also shorten the prompt: `%~` shows `~proj/src`
- `cdpath=(~ ~/code)` lets `cd app` find `~/code/app`

ALIASES & NAVIGATION

```
alias ll='ls -lah'              # regular alias
alias -g G='| grep -i'          # global: works anywhere
ps aux G python                 # → ps aux | grep -i python
alias -g NE='2>/dev/null'       # find / -name x NE
alias -s md=code                # suffix: typing notes.md opens it in code
unalias ll                      # remove an alias

setopt AUTO_CD AUTO_PUSHD PUSHD_IGNORE_DUPS
src                             # AUTO_CD: same as cd src
..                              # up one level
cd -                            # previous directory
dirs -v                         # numbered directory stack
cd -2                           # jump to stack entry 2 (try cd -<Tab>)
hash -d proj=~/code/myproject # create a named directory
cd ~proj                        # → ~/code/myproject
cd /usr/local/bin
cd local share                  # → /usr/share/bin   (swap part of $PWD)
```

CODE EXPLAINED

→ `alias -g` — expands anywhere, so pick names that won't clash (uppercase is common)

→ `alias -s ext=cmd` — `file.ext` alone runs `cmd file.ext`

→ `AUTO_PUSHD` — every `cd` pushes the old dir onto the stack

→ `cd old new` — replaces `old` with `new` in the current path

ALIAS COMMANDS

SYNTAX	EFFECT
`alias n='cmd'`	define
`alias -g N='…'`	global alias
`alias -s ext=cmd`	suffix alias
`alias`	list all
`unalias n`	remove
`\cmd`	skip alias once

DIRECTORY STACK

SYNTAX	EFFECT
`cd -`	previous dir
`dirs -v`	list stack, numbered
`cd -N`	go to entry N
`pushd / popd`	push / pop manually
`AUTO_PUSHD`	cd pushes automatically
`DIRSTACKSIZE=20`	limit stack size

FAST JUMPS

SYNTAX	EFFECT
`AUTO_CD`	`dir` = `cd dir`
`hash -d n=path`	named dir ~n
`cdpath=(~/code)`	search dirs for cd
`cd old new`	swap path part
`cd ~-`	same as previous dir
`z dir`	zoxide / z plugin

⚠ COMMON MISTAKES

- Defining a global alias with a common word: `alias -g e=…` rewrites every standalone `e`
- Expecting aliases inside functions defined *before* the alias: aliases expand when code is read
- Using aliases in scripts: write a function instead
- Forgetting quotes: `alias ll=ls -la` fails. Use `alias ll='ls -la'`

✓ TIPS & BEST PRACTICES

- Use a function when you need arguments in the middle: `mkcd() { mkdir -p $1 && cd $1 }`
- Press Tab after `cd -` to pick from the directory stack menu
- Add `hash -d` names for folders you visit every day
- List global aliases with `alias -g` and suffix aliases with `alias -s`

QUICK SUMMARY

- `alias` / `alias -g` / `alias -s` — 3 kinds
- `AUTO_CD` — cd by name
- `cd -` / `dirs -v` — directory stack
- `hash -d` — named directories
- `cd old new` — path swap

Prompt & Hooks

PROMPTS & HOOKS

The prompt is set with PROMPT (left, alias PS1) and RPROMPT (right side, hidden when the line gets long). It uses **% escapes** for user, host, directory, time and exit status, %F{color} for colors, and %(x.true.false) for conditionals. **Hook functions** run at set moments: precmd before each prompt, preexec before each command, chpwd after a directory change. Register them with add-zsh-hook.

NOTES

- Single-quote prompts so $vars expand each time (with PROMPT_SUBST)
- print -P '…' previews escapes
- Colors: names (red) or 0–255 numbers (%F{208})
- vcs_info ships with zsh and shows git branch info

BUILDING A PROMPT

```
PROMPT='%n@%m %~ %# '          # ada@mac ~/src %
PROMPT='%B%1~%b %# '           # bold, current folder name only
RPROMPT='%*'                   # right side: 14:03:59
PROMPT='%F{cyan}%~%f %(?.%F{green}.%F{red})❯%f '   # red ❯ after a failure

autoload -Uz vcs_info add-zsh-hook
zstyle ':vcs_info:git:*' formats ' (%b)'           # (main)
add-zsh-hook precmd vcs_info  # refresh before every prompt
setopt PROMPT_SUBST           # expand $vars inside PROMPT
PROMPT='%~${vcs_info_msg_0_} %# '                  # ~/src/app (main) %

note_dir() { print "now in $PWD" }
add-zsh-hook chpwd note_dir   # runs after every cd
print -P '%F{208}orange%f %B bold%b'   # preview escapes
```

CODE EXPLAINED

- → %~ — current dir with ~ for home; %1~ — last folder only
- → %(?.A.B) — A if the last command succeeded, else B
- → vcs_info — fills $vcs_info_msg_0_; %b in formats is the branch
- → add-zsh-hook — adds to a hook without replacing other plugins' hooks

PROMPT ESCAPES

ESCAPE	SHOWS
%n %m	user, short host
%~ %1~	dir, last dir
%#	# if root, else %
%?	last exit status
%* %D{%H:%M}	time, custom date
%j	number of jobs

STYLING

ESCAPE	EFFECT
%F{red} … %f	text color
%K{blue} … %k	background color
%B … %b	bold
%U … %u	underline
%(?.a.b)	if last cmd ok
%(!.a.b)	if root

HOOK FUNCTIONS

HOOK	RUNS
precmd	before each prompt
preexec	before each command
chpwd	after cd
periodic	every $PERIOD secs
zshexit	when the shell exits
zshaddhistory	before saving history

⚠ COMMON MISTAKES

- Double-quoting a prompt with $vars: they expand once, when .zshrc loads
- Forgetting PROMPT_SUBST: ${vcs_info_msg_0_} shows literally
- Defining precmd() { … } directly: it overwrites hooks set by plugins. Use add-zsh-hook
- Using raw ANSI codes without %{ %}: zsh miscounts the width and the cursor jumps

✓ TIPS & BEST PRACTICES

- Show the exit code only on failure: %(?..%F{red}[%?]%f)
- Use %1~ or %3~ to keep long paths short
- Put slow info (like git status) in RPROMPT or use an async prompt
- Use preexec + precmd with $EPOCHREALTIME (zsh/datetime) to time commands

QUICK SUMMARY

- PROMPT / RPROMPT — left / right
- %~ %n %m %# — core escapes
- %F{c}…%f — color
- %(?.a.b) — conditional
- add-zsh-hook precmd fn — hooks

Completion System

COMPLETION SYSTEM

Zsh's completion system (**compsys**) is turned on by `compinit`. It ships completions for hundreds of commands and loads extras from files named `_command` in `$fpath`. Behavior is tuned with **`zstyle`**, which takes a context pattern like `':completion:*'` and a setting. You can write your own completion with `_arguments`, or reuse an existing one with `compdef`.

NOTES

- Add to `fpath` **before** `compinit` runs
- `compinit` caches results in `~/.zcompdump`
- Delete `~/.zcompdump*` if new completions don't appear
- `compaudit` lists insecure completion dirs

SETUP & CUSTOM COMPLETION

```
fpath=(~/.zsh/completions $fpath)              # before compinit
autoload -Uz compinit && compinit
zstyle ':completion:*' menu select             # arrow-key menu
zstyle ':completion:*' matcher-list 'm:{a-z}={A-Z}'   # case-insensitive
zstyle ':completion:*' list-colors ${(s.:.)LS_COLORS}
zstyle ':completion:*:descriptions' format '%F{yellow}-- %d --%f'
zstyle ':completion:*' group-name ''           # group by type
zstyle ':completion:*' use-cache on            # cache slow results

compdef _gnu_generic mytool   # complete --flags from mytool --help
compdef mygit=git             # use git's completion for mygit

# ~/.zsh/completions/_deploy
#compdef deploy
_arguments \
  '-v[verbose output]' \
  '--env=[target environment]:env:(dev staging prod)' \
  '*:file:_files'
```

CODE EXPLAINED

- → `menu select` — Tab opens a menu you can move through with arrows
- → `m:{a-z}={A-Z}` — lowercase typing matches uppercase names
- → `#compdef deploy` — first line of the file; ties it to the command
- → `'-v[…]'` — a flag with a description
- → `:env:(dev staging prod)` — a value chosen from a list

USEFUL ZSTYLES

SETTING	EFFECT
`menu select`	interactive menu
`matcher-list …`	fuzzy / case matching
`list-colors`	colored file lists
`group-name ''`	group by category
`squeeze-slashes true`	a//b → a/b
`use-cache on`	cache results

COMMANDS

COMMAND	EFFECT
`compinit`	start completion
`compinit -C`	skip security check (faster)
`compdef _fn cmd`	assign a completer
`compdef a=b`	copy b's completion
`_gnu_generic`	parse --help output
`Ctrl-X h`	show completion context

_ARGUMENTS SPECS

SPEC	MEANING
`'-f[desc]'`	flag
`'-o[desc]:name:'`	flag with value
`'(-a -b)-c[…]'`	excludes -a, -b
`':arg:(a b)'`	positional from list
`'*:file:_files'`	any number of files
`':host:_hosts'`	built-in completer

⚠ COMMON MISTAKES

- Running `compinit` before changing `fpath`: new completions are ignored
- Calling `compinit` several times (framework + your own): it slows startup
- Leaving a stale `~/.zcompdump`: delete it after adding completion files
- Using `bash` completion scripts directly: load them with `bashcompinit` first

✓ TIPS & BEST PRACTICES

- Many CLIs print their own: `gh completion -s zsh > ~/.zsh/completions/_gh`
- Press Ctrl-X h to see the context name to use in a `zstyle`
- Use `compinit -C` once your setup is stable to skip the audit
- Bind Shift-Tab to go back: `bindkey '^[[Z' reverse-menu-complete`

QUICK SUMMARY

- `compinit` — enable completion
- `zstyle ':completion:*' …` — configure
- `compdef` — assign completers
- `_arguments` — write completions
- `$fpath` — where `_cmd` files live

Plugins, Frameworks & Performance

EXTENDING ZSH

Frameworks like Oh My Zsh bundle settings, themes and plugins in one install. **Plugin managers** like antidote or zinit just download and load the plugins you list. You can also clone a plugin and `source` it yourself. Every plugin adds startup time, so **measure** with `zprof` and `time zsh -i -c exit`, and keep startup under roughly 100–200 ms.

NOTES

- Load `zsh-syntax-highlighting` **last**: it wraps widgets made by other plugins
- Most plugins are plain `.zsh` files you can `source`
- Prompts like Starship work in any shell; Powerlevel10k is zsh-only

LOADING & PROFILING

```
# Oh My Zsh (in ~/.zshrc)
ZSH=$HOME/.oh-my-zsh
plugins=(git z fzf)
source $ZSH/oh-my-zsh.sh

# antidote: repos listed in ~/.zsh_plugins.txt
source ~/.antidote/antidote.zsh
antidote load                # clones, sources, caches a bundle

# No manager: clone once, then source
source ~/.zsh/zsh-autosuggestions/zsh-autosuggestions.zsh
eval "$(starship init zsh)"   # Starship prompt

zmodload zsh/zprof           # FIRST line of .zshrc
zprof                        # LAST line: time spent per function
time zsh -i -c exit          # total interactive startup time
```

CODE EXPLAINED

- → `plugins=(…)` — Oh My Zsh loads each from `$ZSH/plugins`
- → `antidote load` — reads `.zsh_plugins.txt` and builds a static load file
- → `eval "$(starship init zsh)"` — prints the setup code, then runs it
- → `zprof` — the top rows show what to remove or lazy-load

FRAMEWORKS & MANAGERS

TOOL	STYLE
`Oh My Zsh`	full framework, 300+ plugins
`Prezto`	lighter framework
`antidote`	fast, plain-text list
`zinit`	turbo / lazy loading
`sheldon`	TOML config, Rust

POPULAR PLUGINS

PLUGIN	ADDS
`zsh-autosuggestions`	gray history hints
`zsh-syntax-highlighting`	color as you type
`zsh-completions`	extra completions
`fzf`	fuzzy Ctrl-R, Ctrl-T
`zoxide`	z smart cd

PROMPTS & SPEED

TOOL / FIX	EFFECT
`Powerlevel10k`	fast zsh theme, wizard
`Starship`	cross-shell prompt
`compinit -C`	skip completion audit
`lazy-load nvm`	load on first use
`zcompile ~/.zshrc`	precompile to .zwc

⚠ COMMON MISTAKES

- Loading Oh My Zsh **and** a plugin manager that also runs `compinit`: double work at every start
- Enabling dozens of Oh My Zsh plugins you never use: each one costs startup time
- Sourcing syntax highlighting before other widget plugins: highlighting breaks

✓ TIPS & BEST PRACTICES

- Profile first: `zprof` usually points at `nvm`, `compinit`, or a heavy plugin
- Accept an autosuggestion with → or End; bind Ctrl-Space to `autosuggest-accept`
- Try a new plugin in `zsh -f` first, then `source` it, before adding it to `.zshrc`

QUICK SUMMARY

- Oh My Zsh — all-in-one framework
- `antidote` / `zinit` — plugin managers
- `source file.zsh` — manual loading
- `zprof` — find slow startup
- `time zsh -i -c exit` — measure

Quick Reference — Core Syntax

startup files · options · quoting · redirection · history ·

STARTUP & OPTIONS

SYNTAX	DOES	#
`.zshenv`	every shell	01
`.zprofile`	login shells	01
`.zshrc`	interactive shells	01
`zsh -f`	no config	01
`setopt X`	option on	02
`unsetopt X`	option off	02
`emulate -L zsh`	safe local defaults	02
`[[ -o X ]]`	option set?	02

OUTPUT & QUOTING

SYNTAX	DOES	#
`print -l a b`	one per line	03
`print -P '%F{red}'`	prompt escapes	03
`print -u2 msg`	to stderr	03
`printf '%5.2f'`	formatted	03
`'text'`	literal	03
`"$v"`	expands	03
`$'a\tb'`	real escapes	03
`${(q)v}`	quoted value	03

REDIRECTION

SYNTAX	DOES	#
`> f >> f`	write, append	04
`2> f`	stderr	04
`&> f`	stdout + stderr	04
`a |& b`	pipe both	04
`>| f`	force (noclobber)	04
`<<EOF`	here-doc	04
`<<< "$s"`	here-string	04
`$pipestatus`	pipe exit codes	04

HISTORY & KEYS

SYNTAX	DOES	#
`!!`	last command	05
`!$ !*`	last arg, all args	05
`^old^new`	fix and rerun	05
`fc`	edit last command	05
`Ctrl-R`	search history	05
`Alt-.`	insert last word	06
`bindkey -v`	vi mode	06
`zle -N fn`	make a widget	06

VARIABLES & PARAMS

SYNTAX	DOES	#
`typeset -i n`	integer	07
`local x`	function scope	07
`export X=1`	to child processes	07
`typeset -p x`	inspect	07
`$# "$@"`	arg count, args	08
`$?`	last exit code	08
`$$ $!`	shell, job PID	08
`path+=(~/bin)`	edit PATH	08

EXPANSION

SYNTAX	DOES	#
`${v:-x}`	default	09
`${v#p} ${v%p}`	trim front, back	09
`${v//a/b}`	replace all	09
`${v:2:3}`	substring	09
`${(U)v}`	uppercase	10
`${(s:,:)v}`	split	10
`${(j:,:)a}`	join	10
`${(f)"$(c)"}`	lines	10

ARRAYS

SYNTAX	DOES	#
`a=(x y z)`	create	11
`$a[1] $a[-1]`	first, last	11
`$a[2,3]`	slice	11
`a+=(w)`	append	11
`${#a}`	length	11
`${a[(Ie)x]}`	index, 0 if none	11
`${a:|b}`	difference	11

ASSOCIATIVE ARRAYS

SYNTAX	DOES	#
`typeset -A h`	declare	12
`h[k]=v`	set	12
`$h[k]`	get	12
`${(k)h} ${(v)h}`	keys, values	12
`${+h[k]}`	key exists?	12
`${(ok)h}`	sorted keys	12
`unset 'h[k]'`	delete key	12

HANDY PARAMETERS

SYNTAX	DOES	#
`$0`	script / function	08
`${10}`	arg 10+	08
`$argv`	args as array	08
`$PWD $OLDPWD`	dirs	08
`$RANDOM`	0–32767	08
`$SECONDS`	shell uptime	08
`$ZSH_VERSION`	version	01

⚠ COMMON MISTAKES

- `$arr[0]` is empty: arrays start at 1
- `name = value` runs a command; no spaces around `=`
- Unquoted URLs fail with `no matches found`

✓ TIPS & BEST PRACTICES

- `print -l` anything to see it one item per line
- `typeset -p` shows a variable's type and value
- `emulate -L zsh` at the top of every function

QUICK SUMMARY

- Number column = sheet with full details
- `man zshall` — the complete manual
- Continued on Sheet 27

Quick Reference — Advanced

GLOBBING

SYNTAX	DOES	#
`**/*.py`	recursive	13
`***/`	follow symlinks	13
`<1-10> <->`	number ranges	13
`(a|b)`	alternatives	13
`{1..5} {a,b}`	brace expansion	13
`^*.bak`	not (EXT_GLOB)	13
`*.log~err*`	except (EXT_GLOB)	13
`(#i)readme*`	ignore case	13

GLOB QUALIFIERS

SYNTAX	DOES	#
`*(.) *(/)`	files, dirs	14
`*(@) *(*)`	links, executables	14
`*(om[1])`	newest	14
`*(OL[1,5])`	5 largest	14
`*(Lm+100)`	over 100 MB	14
`*(mh-1)`	changed < 1 hour	14
`*(D)`	include dotfiles	14
`*(N)`	no error if none	14

MODIFIERS & TESTS

SYNTAX	DOES	#
`$f:h $f:t`	dirname, basename	14
`$f:r $f:e`	no ext, ext	14
`$f:A`	real absolute path	14
`[[ -f f ]]`	is a file	15
`[[ -z $s ]]`	empty string	15
`[[ $s == p* ]]`	pattern match	15
`[[ $s =~ re ]]`	regex, $match	15
`(( a > b ))`	number compare	15

BRANCHING & MATH

SYNTAX	DOES	#
`if …; then … fi`	branch	16
`case $x in p) ;;`	match patterns	16
`;& ;|`	fall through, retest	16
`select x in …`	menu	16
`$(( 7 / 2.0 ))`	float math: 3.5	17
`(( x++ ))`	math statement	17
`$(( [#16] 255 ))`	hex: 16#FF	17
`zmodload zsh/mathfunc`	sqrt, floor…	17

LOOPS

SYNTAX	DOES	#
`for x in …; do`	list loop	18
`for x ($a) cmd`	short form	18
`for ((i=0;i<n;i++))`	counter	18
`for k v in …`	2 vars per pass	18
`while read -r l`	read lines	18
`until cmd`	loop until success	18
`repeat 3 cmd`	fixed count	18
`break / continue N`	loop control	18

FUNCTIONS & SCRIPTS

SYNTAX	DOES	#
`f() { … }`	define	19
`() { … } arg`	anonymous	19
`autoload -Uz f`	load from $fpath	19
`setopt ERR_EXIT`	stop on error	20
`trap '…' EXIT`	cleanup	20
`{ } always { }`	try / finally	20
`zparseopts`	parse flags	20
`zsh -x f.zsh`	trace	20

JOBS & ALIASES

SYNTAX	DOES	#
`cmd &!`	background, disowned	21
`jobs fg bg`	job control	21
`diff <(a) <(b)`	output as file	21
`vim =(cmd)`	temp file	21
`alias -g G='|grep'`	global alias	22
`alias -s md=code`	suffix alias	22
`dirs -v; cd -2`	dir stack	22
`hash -d p=~/code`	named dir ~p	22

PROMPT & HOOKS

SYNTAX	DOES	#
`%~ %1~`	dir, last dir	23
`%n %m %#`	user, host, %/#	23
`%F{red}…%f`	color	23
`%B…%b`	bold	23
`%(?.ok.err)`	on exit status	23
`PROMPT_SUBST`	expand $vars	23
`vcs_info`	git branch	23
`add-zsh-hook`	precmd, chpwd…	23

COMPLETION & PLUGINS

SYNTAX	DOES	#
`compinit`	enable completion	24
`menu select`	arrow menu (zstyle)	24
`compdef a=b`	reuse completion	24
`_arguments`	write completion	24
`plugins=(…)`	Oh My Zsh	25
`antidote load`	plugin manager	25
`zprof`	profile startup	25
`zsh -i -c exit`	time startup	25

⚠ COMMON MISTAKES

- No match is an error: add `(N)` to globs that may be empty
- `EXTENDED_GLOB` is needed for `^`, `~`, `#`
- `$(( 1 / 3 ))` is `0`; use `1.0 / 3`

✓ TIPS & BEST PRACTICES

- Preview risky globs with `print -l` before `rm`
- `zsh -f` isolates config problems
- `time zsh -i -c exit` keeps startup honest

QUICK SUMMARY

- Number column = sheet with full details
- `man zshexpn` — expansion & globbing
- `man zshoptions` — every option